1 Project Details

Team Name / Number

Team 115

Team Members

- Nicole Kim
- Karen Lee
- Renny Matis
- Bilal Sadozai
- Lu Yang

Project title

Student Attrition Risk & Intervention Briefing

Project Owner / Client

QUT Digital Business Solutions – Data & Analytics

Project Type

Development Project

Project Artefact

Development of an AI-Driven Student Attrition Risk and Early Intervention Solution Using Databricks. The proposed artefact integrates synthetic student data, machine learning-based attrition risk prediction, and AI-generated intervention briefings into an application designed for academic advisors. The solution is designed to enable Academic Advisors to identify at-risk students, assess contributing risk factors, and determine appropriate early intervention strategies.

2 Artefact Deliverables & Scope

Section 2 defines the Artefact Deliverables & Scope for the project regarding the primary deliverables of synthetic data generation, a machine learning model, and a fully developed software application on Databricks.

Below details the Artefact Deliverables & Scope pertaining to the synthetic data generation and machine learning model:

Data Generation Supplementary Data

- .csv files that hold distribution data on real data for synthetic data generation

- Student schema of real data that provides the base structure for synthetic data

Synthetic Data Generation Code Notebooks

- Python code notebooks (.ipynb) that generate synthetic data by taking information from the distribution data and student schema and creating 12 data tables
- Data tables themselves will not be handed over as they are large and difficult to download or transfer

Data Validation Documentation and Notebooks

- Reports and result spreadsheet files on data validation performed on the synthetic data, and statistical validation performed on the beta distribution using both the source data (publicly available attrition rates) and synthetic data (data generated using beta distribution to determine attrition rate distribution).

Machine Learning Model Code Notebook

- Python code notebook (.ipynb) that builds and trains a Random Forest classifier that can identify students that are at a high risk of attrition. Results are recorded in data tables.
- Data tables themselves will not be handed over as they are large and difficult to download or transfer

Machine Learning Model Documentation

- Documentation of everything that was used during the development of the machine learning model
    - AI prompts used
    - Implementation traceability – assumptions and limitations; feature selection, exclusion, and importance; amendment history; outputs
    - Spec-Driven Documentation

Below details the Artefact Deliverables & Scope pertaining to the fully developed Databricks application:

Databricks Application and Dashboard

- Final advisor facing Databricks application and dashboard implementing the agreed application functionality of allowing an Academic Advisor to view a dashboard that displays students identified as being at risk and allowing for the request of a Structured Advisor Briefing.

Application Backend

- Functionality of the Databricks application backend to enable the application's dashboard, student selection, application interactions and Advisor briefing requests, this is done through utilising delta tables of synthetic data generation and machine learning model outputs as data to support the application's use case.

Application Source Code

- Final source code for Databricks application (inclusive of dashboard, backend, and application functionality) will be provided in the project GitHub repository.

Github and Databricks Integration

- Code will be completed and documented with version-control integration between Github Repository and Databricks development environment, providing an accurate view of historical and traceable development of the application.

End-to-End Application Integration

- Separately developed project components will be integrated into the Databricks application (Synthetic data gen and ML model as project components).

Application Testing, Refinement and Defect Resolution

- Testing and refining the application and dashboard and resolving identified application defects within the project scope to ensure a working application.

Technical Documentation

- Technical documentation documented within notebook development files and GitHub repository for application architecture, implementation, integration, analysis and limitations.

3 User Stories, Requirements & Acceptance

This section identifies the application, integration and Generative AI user stories that define the agreed system functionality and explains how successful delivery will be demonstrated. The project user stories maintained in GitHub provide traceability between requirements, sprint implementation, testing, and final artefact delivery.

The application will consume the machine-learning prediction outputs and relevant synthetic student data stored in Databricks Delta tables. The student_deidentified_hash will be used to associate a selected student with their attrition-risk prediction and relevant student information.

User Story / Requirement	Acceptance Criteria	Evidence of Delivery
US-08 – Application Backend	The application backend can retrieve required student and risk information from the project Delta tables and support the advisor-facing workflow.	Demonstrate the application retrieving real project data rather than hard-coded test values.
US-09 – Application / Dashboard Frontend	The advisor-facing interface displays the required student-risk information and provides access to the agreed briefing functionality in a usable layout.	Demonstrate the final Databricks dashboard/application and its main interactions.
US-10 – At-Risk Student Display	Students meeting the agreed attrition-risk threshold are displayed with their	Compare displayed student-risk information with the

	deidentified identifier, risk percentage, and risk status.	corresponding ML prediction records.
US-11 – Student Selection and Briefing Request	When an advisor selects an at-risk student and requests a briefing, the application retrieves the corresponding student information and initiates the briefing workflow.	Select a synthetic student and demonstrate retrieval of the correct prediction and associated student context.
US-12 – Structured Briefing Instructions	The application uses agreed briefing instructions together with relevant student and prediction information when preparing the Generative AI request.	Demonstrate that the application combines the agreed briefing instructions with the selected student's retrieved context and risk prediction before generating the briefing.
US-13 – Generative AI Briefing Generation	A compatible Generative AI model generates a Structured Advisor Briefing using the selected student information and attrition-risk result. AI interpretations are presented as interpretations rather than confirmed causation.	Generate and display a briefing for a selected synthetic student through the application.
US-14 – Risk Factor Presentation	The application provides Academic Advisor with relevant student information and key factors contributing to the selected student's predicted attrition risk in a clear and accessible format.	Select an at-risk student and confirm that the displayed risk factors and supporting information match the student's data and predicted attrition risk.
US-15 – Human Review and Decision Support	The Structed Advisor Briefing provides decision-support information while clearly separating AI-generated interpretations from verified information, with Academic Advisors remaining responsible for all intervention decisions.	Demonstrate a generated briefing that clearly identifies AI-generated content and recommendations for the review by the Academic Advisor.
US-16 – End-to-End Integration	The ML prediction, application and Generative AI components operate together as an integrated workflow.	Demonstrate the complete flow from an ML prediction record through student selection to generation of a Structured Advisor Briefing.
US-17–Briefing Storage & Retrieval	Generate Structured Advisor Briefings can be stored and accessed for the relevant deidentified student.	Generate and store a briefing, then access the saved briefing for the same student through the application.
US-18 – Search & Risk Filtering	Academic Advisors can locate students and refine the at-risk student list using defined criteria, such as student identifier or risk classification.	Demonstrate the search and filtering features and confirm that the displayed records align with the specified criteria.

US-19 – Advisor Notes	The application enables Academic Advisors to add notes related to a student's briefing or intervention activities.	Record and save an advisor note, then verify that the stored note is correctly associated with the relevant student record and briefing.
US-20 – Error Handling	If required student information, prediction results, or AI-generated outputs are unavailable; the application provides a clear error or data-unavailable message instead of presenting incomplete or potentially misleading information.	Test at least one controlled failure scenario and verify that the application displays an appropriate message to the user,
US-21 – Deidentified Student Handling	Student records within the application are linked using student_deidentified_hash, allowing the prototype to operate without the use of directly identifiable student information.	Track a selected synthetic student across the relevant Delta tables, model results, and application using the deidentified identifier.
US-22 – Briefing Traceability	The briefing clearly reflects the selected student's retrieved context and corresponding ML prediction rather than generic or unrelated information.	Verify that the generated briefing aligns with the selected student's source data and prediction results.

Application and ML Integration

The primary integration flow will be:

Synthetic Data
↓
Machine Learning Model
↓
Student Attrition Risk Prediction Delta Table
↓
Databricks Dashboard / Application
↓
Select At-Risk Student
↓
Retrieve Student + Risk Information
↓
Generative AI Model
↓
Structured Advisor Briefing

The application will consume the ML output rather than reproduce the prediction process. Subject to the capabilities and model availability of the Databricks Free Edition environment, the exact Generative AI model may vary according to the models available and suitable within the Databricks environment.

Briefing Validation, Storage and MCP

Additional functionality such as briefing validation, regeneration, persistent briefing storage and MCP/agent integration may be implemented where included in the agreed Base, Enhanced or Stretch scope.

Where implemented, these capabilities will be tested against their corresponding user stories and acceptance criteria. Their implementation should not prevent delivery of the core workflow covering risk predication, student selection, student context retrieval and display, and Generative AI briefing generation.

MCP may be used to expose selected Databricks data or functionality to an MCP-compatible client or AI agent but is not required for communication between the Databricks application and its own Delta tables. Where included in the agreed scope, MCP may be implemented using Databricks managed capabilities or deployed as part of, or separately from, the Databricks application depending on the final technical requirements and agreed scope.

Sprint and Acceptance Approach

Application functionality will be developed progressively across the remaining sprints, beginning with real Delta-table integration and student selection, followed by Generative AI integration and final end-to-end testing.

Successful delivery of the application/AI scope will be demonstrated through traceable evidence that the agreed user stories have been implemented, tested and demonstrated, with the primary acceptance scenario showing an Academic Advisor moving from an at-risk student prediction to a generated Structured Advisor Briefing.

4 Base / Enhanced / Stretched Scopes

Section 4 – Base / Enhanced / Stretch Scope

Scope Overview

The agreed system functionality is grouped into Base, Enhanced, and Stretch scope levels. Each level builds upon the previous level and distinguishes the core functionality the team commits to deliver from additional functionality that may be completed if time and resources permit. The scope remains aligned with the team's approved solution architecture and the artefact deliverables defined in Section 2.

Base Scope – Core Deliverable

The Base Scope represents the minimum viable end-to-end solution that the team commits to delivering.

Synthetic Data Generation

- Distribution .csv files for synthetic data generation.
- Student schema defining the synthetic data structure.
- Python notebooks for synthetic data generation.
- Validation documentation and statistical validation notebooks.

Machine Learning

- Random Forest machine learning model for student attrition prediction.
- Machine learning development documentation, including implementation traceability and Spec-Driven documentation.

Databricks Application

- Advisor-facing Databricks application with Student Risk Overview.
- AI-generated Structured Advisor Briefing.
- Application backend supporting dashboard interactions and briefing requests.
- End-to-end integration between synthetic data generation, ML outputs and the application.
- Application source code within the GitHub repository.
- GitHub–Databricks version control integration.
- Application testing, refinement and defect resolution to deliver a working application.

Supporting Deliverables

- Technical documentation covering architecture, implementation and integration.
- GitHub repository containing project artefacts and source code.

Base Outcome

A complete working proof of concept that demonstrates the full workflow from synthetic data generation through to AI-assisted advisor decision support.

Enhanced Scope

The Enhanced Scope includes functionality that improves usability, quality and transparency beyond the committed core deliverables.

- Improved dashboard usability and navigation.
- Enhanced visualisation of student risk indicators.
- More detailed AI-generated advisor briefings.
- User documentation explaining application usage.
- Additional refinement of prediction outputs.
- Further refinement of technical documentation.

Enhanced Outcome

An improved user experience with stronger documentation and more polished decision-support functionality.

Stretch Scope

The Stretch Scope includes aspirational functionality that may be pursued after the Base and Enhanced scopes are completed.

- Interactive dashboard filtering and drill-down functionality.
- Advanced AI-generated intervention recommendations.
- Additional analytics and reporting capabilities.
- Expanded explainability of prediction results.
- Greater automation of advisor briefing generation.
- Additional workflow improvements within the Databricks environment.

Stretch Outcome

Advanced functionality that extends the existing architecture without changing the agreed core system.

Scope Alignment with Solution Architecture

The agreed scope aligns directly with the team's conceptual and logical solution designs.

Revised Scope Alignment with Solution Architecture

Architecture Component / Deliverable	**Scope Level**
Distribution .csv files	Base
Student Schema	Base
Synthetic Data Generation Notebooks	Base
Synthetic Data Validation Documentation	Base
Random Forest ML Model	Base
ML Development Documentation	Base
Delta Tables	Base
Databricks Application	Base
Student Risk Overview Dashboard	Base
AI-Generated Structured Advisor Briefing	Base
Application Backend	Base
Application Source Code	Base

GitHub–Databricks Integration	Base
End-to-End Application Integration	Base
Application Testing, Refinement & Defect Resolution	Base
Technical Documentation	Base
Improved Dashboard Usability	Enhanced
Enhanced Risk Visualisations	Enhanced
User Documentation	Enhanced
Advanced AI Recommendations	Stretch
Interactive Dashboard Features	Stretch
Extended Analytics & Reporting	Stretch
Greater Briefing Automation	Stretch

5 Artefact Handover

The Artefact Handover section details all deliverables for the project owner to be handed over at the end of the project, allowing for following up on the project for further development or usage without contacting the development team after the resolution of the project. This includes software, source code, and documentation to understand or continue the project.

Below details what Artefacts will be handed over:

- All development source code handed over through the Github repository, source code for synthetic data generation, the machine learning model, Generative AI components and the Databricks application.
- Supporting technical documentation noted in source code through the Github repository, documentation relating to synthetic data generation, the machine learning model, Generative AI components, and Databricks application.
- Documentation of key solutions, design decisions, and evidence provided through Github and final packaged artefact submission (zip file).
- README documentation in Github to summarise the project and relevant sections (components, supporting documentation).
- Databricks shared team workspace account containing relevant project notebook and finalised Databricks application.
- Project owner provided appropriate access to Github repository and Databricks workspace.

- **Project artefacts handed over before Week 13 Project Artefact Submission deadline (Sunday, 25th October 2026 11:59pm).**
- Handover completed through Github Repository, Databricks workspace, and finalised packaging of artefact/documentation files, providing completed solution (technical implementation files) and supporting materials (documentation).